1. **Import the dataset and do the usual exploratory analysis steps like checking the structure and characteristics of the dataset:**

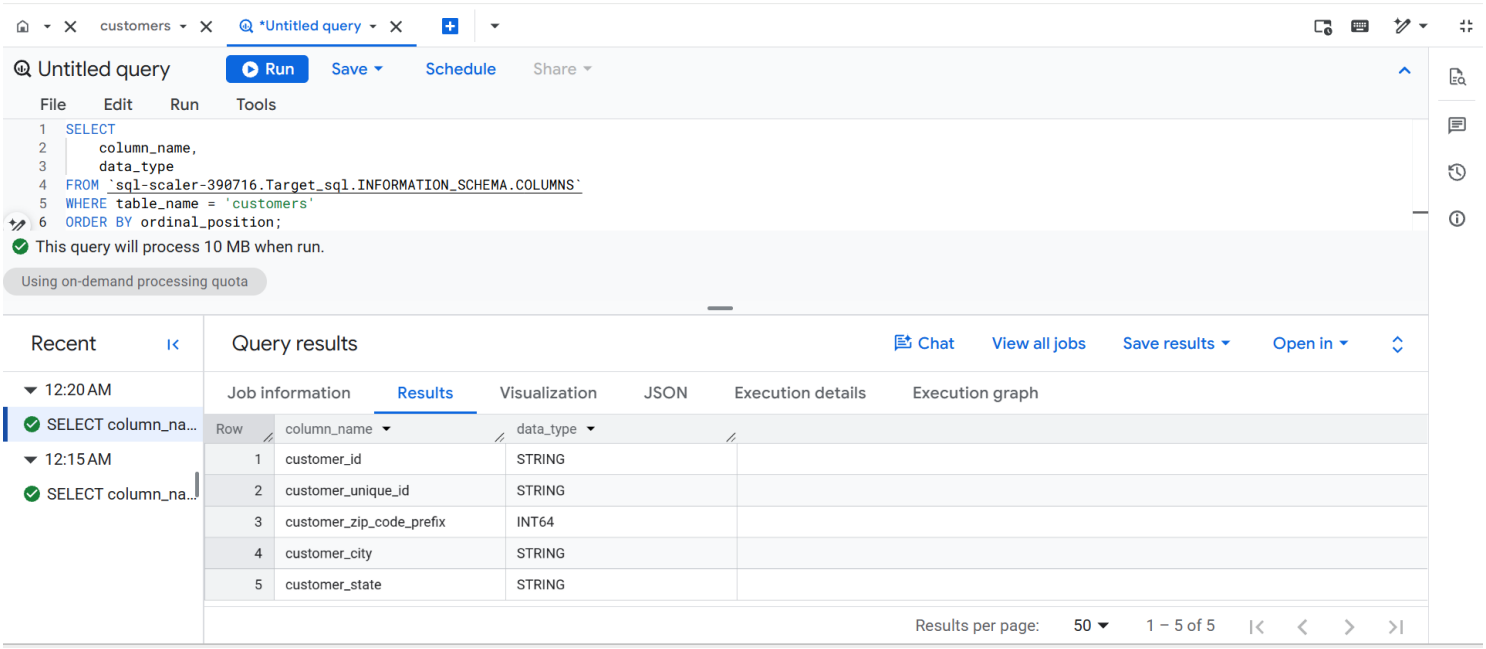
   a)Data type of all columns in the "customers" table.

   b)Get the time range between which the orders were placed.

   c)Count the Cities and states of customers who ordered during the given period.

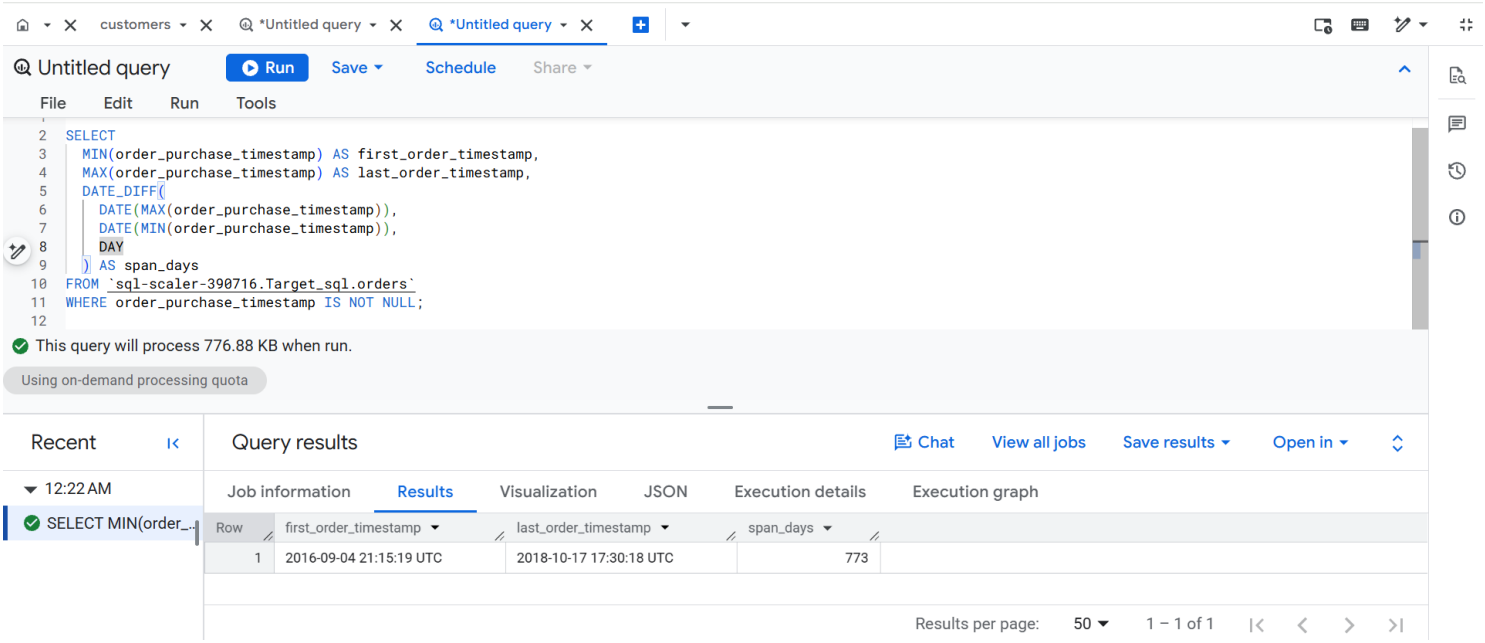
1a) Query:

```
SELECT
  column_name,
  data_type
FROM `sql-scaler-390716.Target_sql.INFORMATION_SCHEMA.COLUMNS`
WHERE table_name = 'customers'
ORDER BY ordinal_position;
```



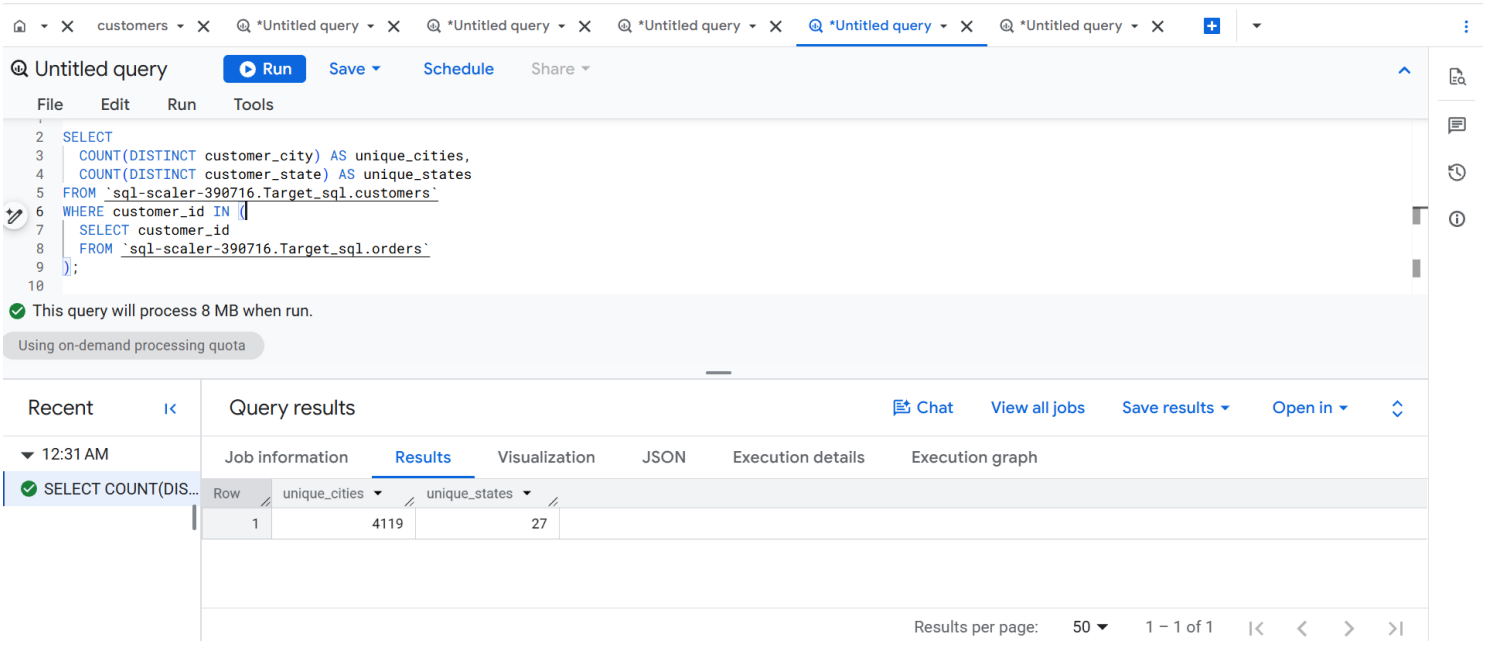
1b) Query:

```
SELECT
  MIN(order_purchase_timestamp) AS first_order_timestamp,
  MAX(order_purchase_timestamp) AS last_order_timestamp,
  DATE_DIFF(
    DATE(MAX(order_purchase_timestamp)),
    DATE(MIN(order_purchase_timestamp)),
    DAY
  ) AS span_days
FROM `sql-scaler-390716.Target_sql.orders`
WHERE order_purchase_timestamp IS NOT NULL;
```



1c) Query:

```
SELECT
 COUNT(DISTINCT customer_city) AS unique_cities,
 COUNT(DISTINCT customer_state) AS unique_states
FROM `sql-scaler-390716.Target_sql.customers`
WHERE customer_id IN (
 SELECT customer_id
 FROM `sql-scaler-390716.Target_sql.orders`
);
```

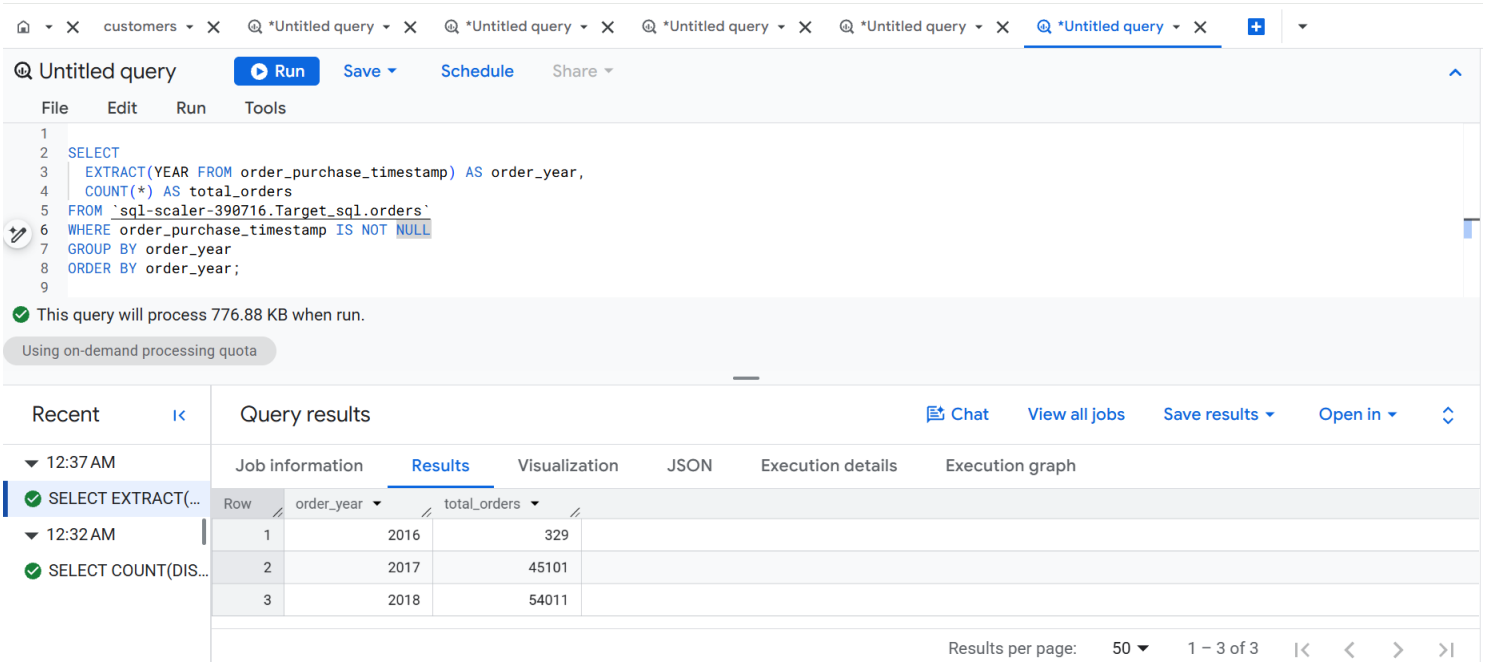


2. **In-depth Exploration:**

1. Is there a growing trend in the no. of orders placed over the past years?
2. Can we see some kind of monthly seasonality in terms of the no. of orders being placed?
3. During what time of day do the Brazilian customers mostly place their orders? (Dawn, Morning, Afternoon or Night)
    - 0-6 hrs: Dawn
    - 7-12 hrs: Mornings
    - 13-18 hrs: Afternoon
    - 19-23 hrs: Night

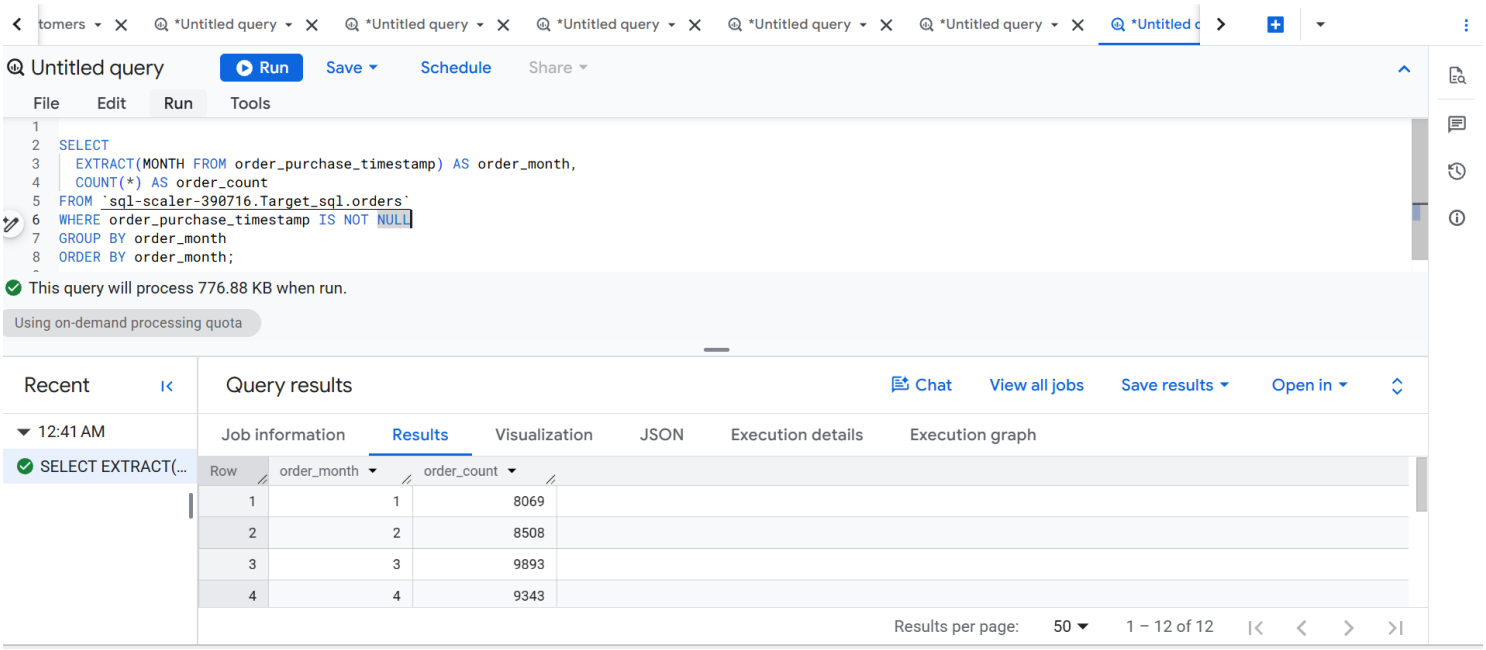
2a) Query:

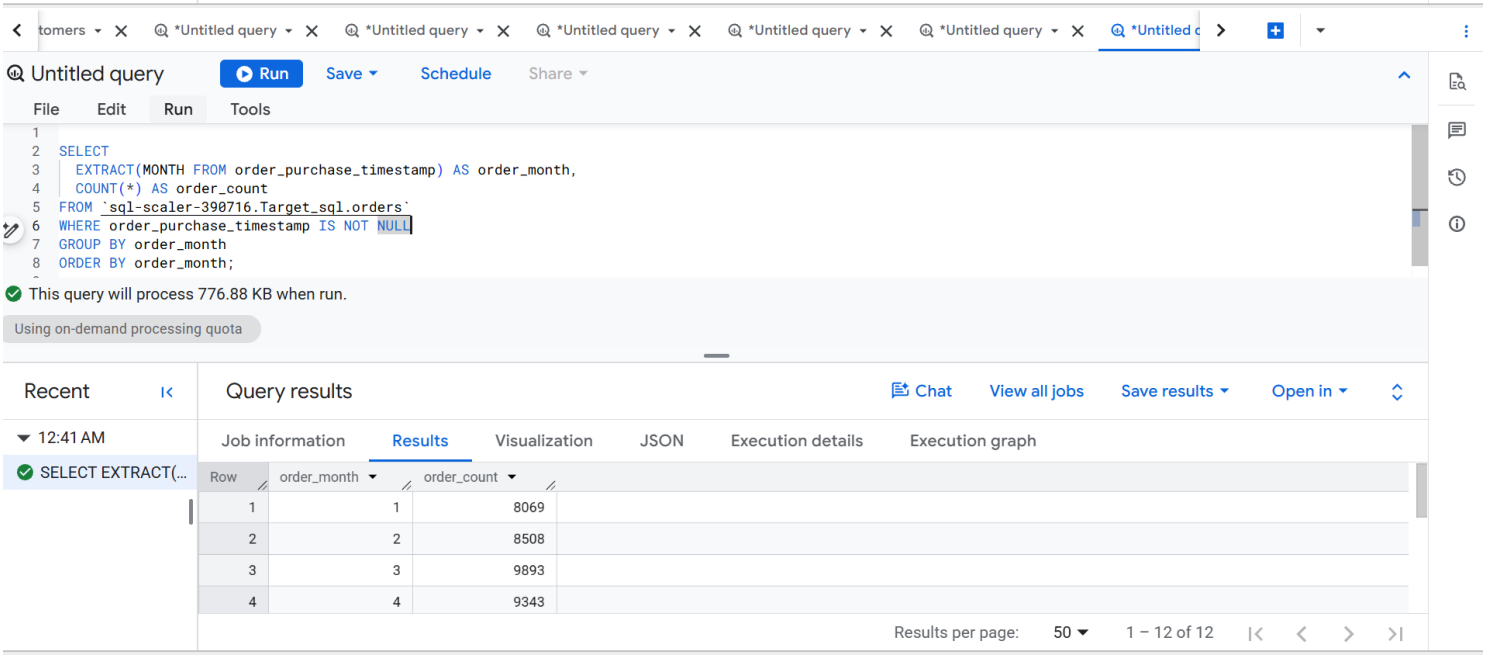
```
SELECT
EXTRACT(YEAR FROM order_purchase_timestamp) AS order_year,
 COUNT(*) AS total_orders
FROM `sql-scaler-390716.Target_sql.orders`
WHERE order_purchase_timestamp IS NOT NULL
GROUP BY order_year
ORDER BY order_year;
```



2b)Query:

```
SELECT
EXTRACT(MONTH FROM order_purchase_timestamp) AS order_month,
COUNT(*) AS order_count
FROM `sql-scaler-390716.Target_sql.orders`
WHERE order_purchase_timestamp IS NOT NULL
GROUP BY order_month
ORDER BY order_month;
```

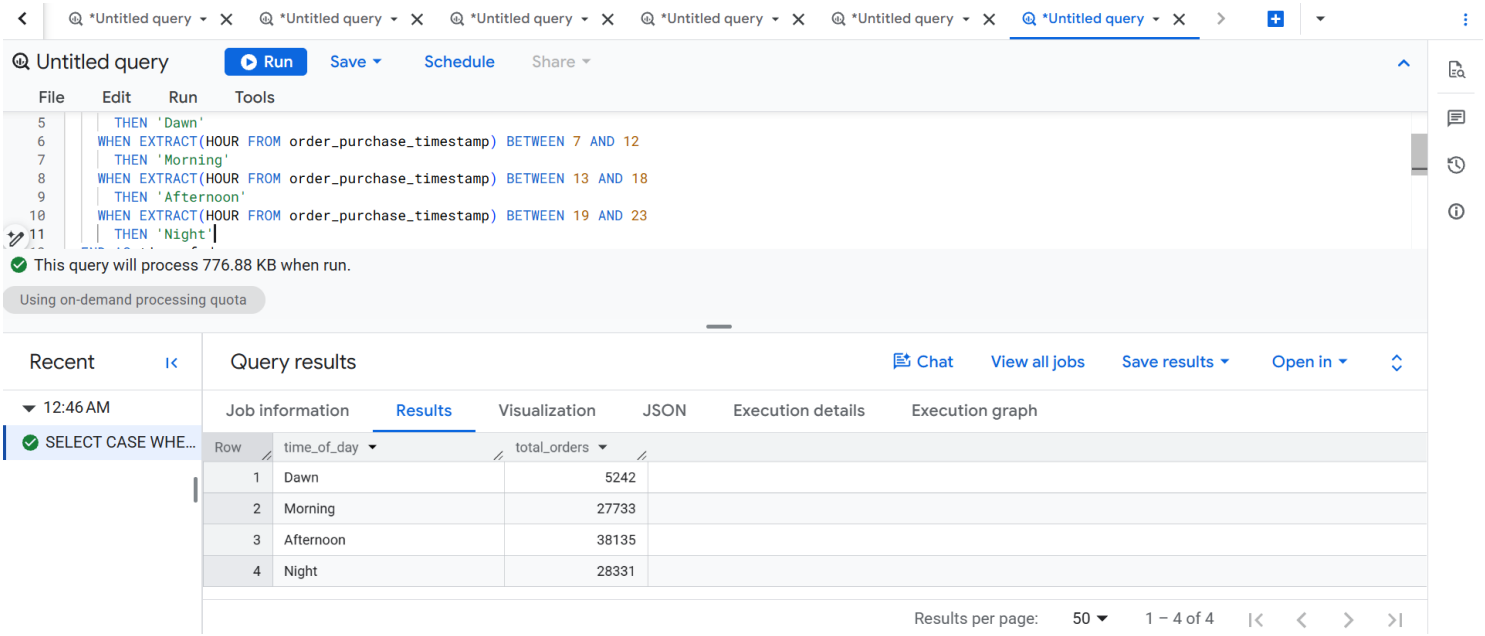




2c) Query:

```
SELECT
  CASE
    WHEN EXTRACT(HOUR FROM order_purchase_timestamp) BETWEEN 0 AND 6
      THEN 'Dawn'
    WHEN EXTRACT(HOUR FROM order_purchase_timestamp) BETWEEN 7 AND 12
      THEN 'Morning'
    WHEN EXTRACT(HOUR FROM order_purchase_timestamp) BETWEEN 13 AND 18
      THEN 'Afternoon'
    WHEN EXTRACT(HOUR FROM order_purchase_timestamp) BETWEEN 19 AND 23
      THEN 'Night'
  END AS time_of_day,
  COUNT(*) AS total_orders
FROM `sql-scaler-390716.Target_sql.orders`
WHERE order_purchase_timestamp IS NOT NULL
GROUP BY time_of_day
ORDER BY
  CASE time_of_day
```

```
    WHEN 'Dawn' THEN 1
    WHEN 'Morning' THEN 2
    WHEN 'Afternoon' THEN 3
    WHEN 'Night' THEN 4
  END;
```



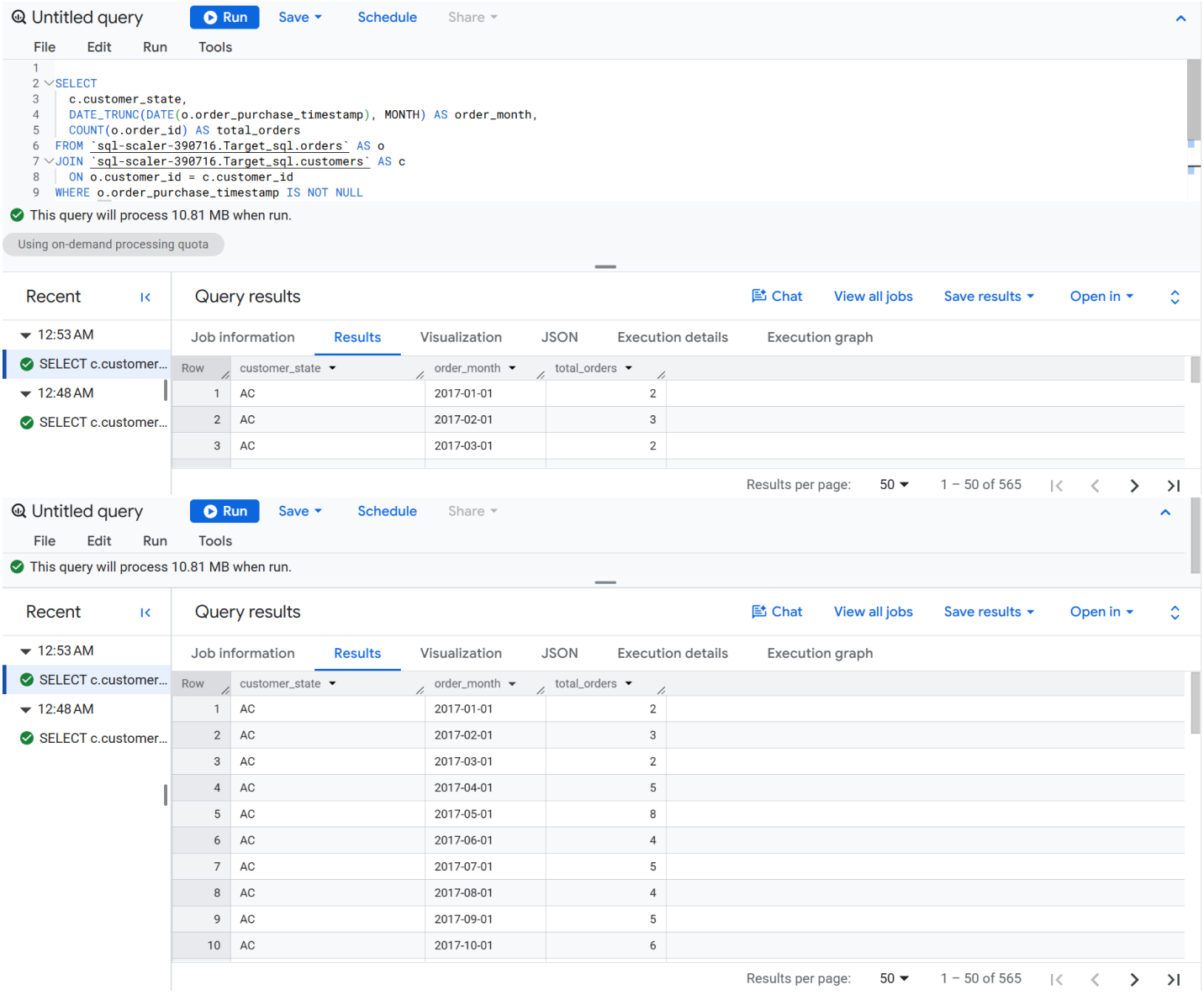
3. **Evolution of E-commerce orders in the Brazil region:**

   a) Get the month-on-month no. of orders placed in each state.

   b)How are the customers distributed across all the states?

3a) Query:

```
SELECT
  c.customer_state,
  DATE_TRUNC(DATE(o.order_purchase_timestamp), MONTH) AS order_month,
  COUNT(o.order_id) AS total_orders
FROM `sql-scaler-390716.Target_sql.orders` AS o
JOIN `sql-scaler-390716.Target_sql.customers` AS c
  ON o.customer_id = c.customer_id
WHERE o.order_purchase_timestamp IS NOT NULL
GROUP BY
  c.customer_state,
  order_month
ORDER BY
  c.customer_state,
  order_month;
```

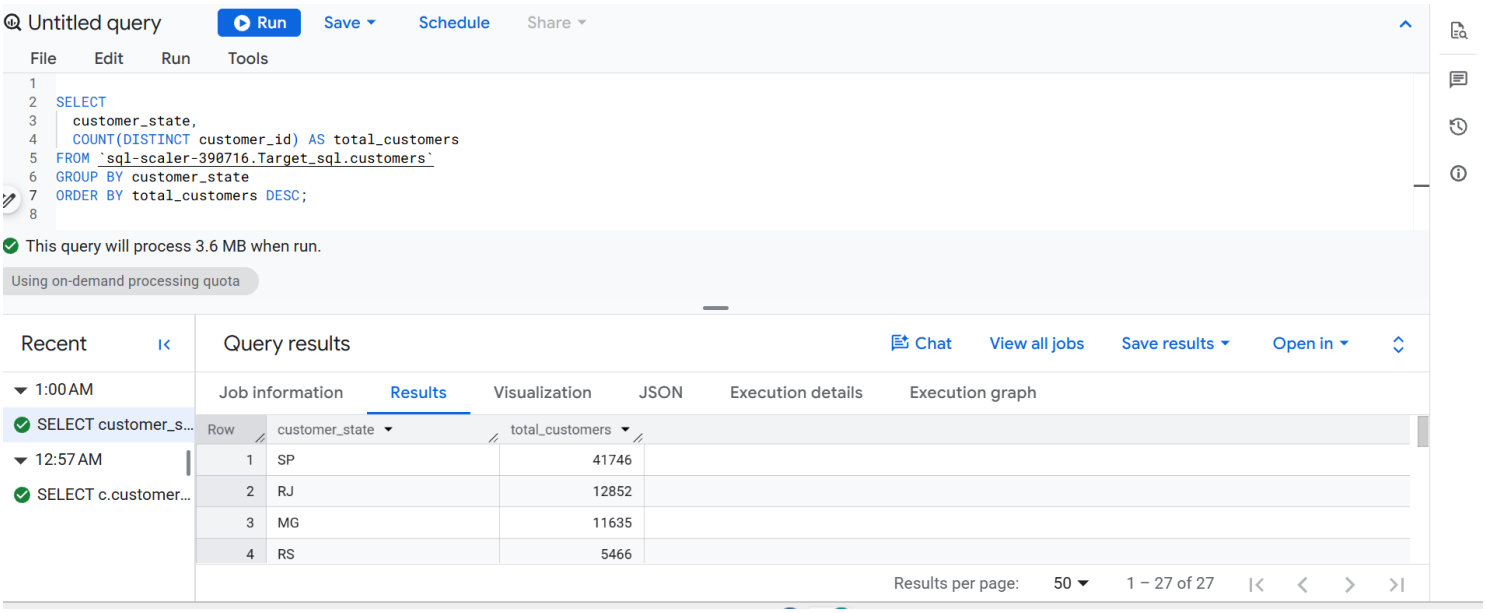


Row	customer_state	order_month	total_orders
1	AC	2017-01-01	2
2	AC	2017-02-01	3
3	AC	2017-03-01	2
4	AC	2017-04-01	5
5	AC	2017-05-01	8
6	AC	2017-06-01	4
7	AC	2017-07-01	5
8	AC	2017-08-01	4
9	AC	2017-09-01	5
10	AC	2017-10-01	6
11	AC	2017-11-01	5
12	AC	2017-12-01	5
13	AC	2018-01-01	6
14	AC	2018-02-01	3
15	AC	2018-03-01	2
16	AC	2018-04-01	4
17	AC	2018-05-01	2
18	AC	2018-06-01	3
19	AC	2018-07-01	4
20	AC	2018-08-01	3
21	AL	2016-10-01	2
22	AL	2017-01-01	2
23	AL	2017-02-01	12
24	AL	2017-03-01	10
25	AL	2017-04-01	23
26	AL	2017-05-01	27

27	AL	2017-06-01	10
28	AL	2017-07-01	17
29	AL	2017-08-01	18
30	AL	2017-09-01	20
31	AL	2017-10-01	28
32	AL	2017-11-01	26

3b) Query:

```
SELECT
  customer_state,
  COUNT(DISTINCT customer_id) AS total_customers
FROM `sql-scaler-390716.Target_sql.customers`
GROUP BY customer_state
ORDER BY total_customers DESC;
```



Output :

Row	customer_state	total_customers
1	SP	41746
2	RJ	12852
3	MG	11635
4	RS	5466
5	PR	5045
6	SC	3637
7	BA	3380
8	DF	2140
9	ES	2033
10	GO	2020
11	PE	1652
12	CE	1336
13	PA	975
14	MT	907
15	MA	747
16	MS	715
17	PB	536
18	PI	495
19	RN	485
20	AL	413
21	SE	350
22	TO	280

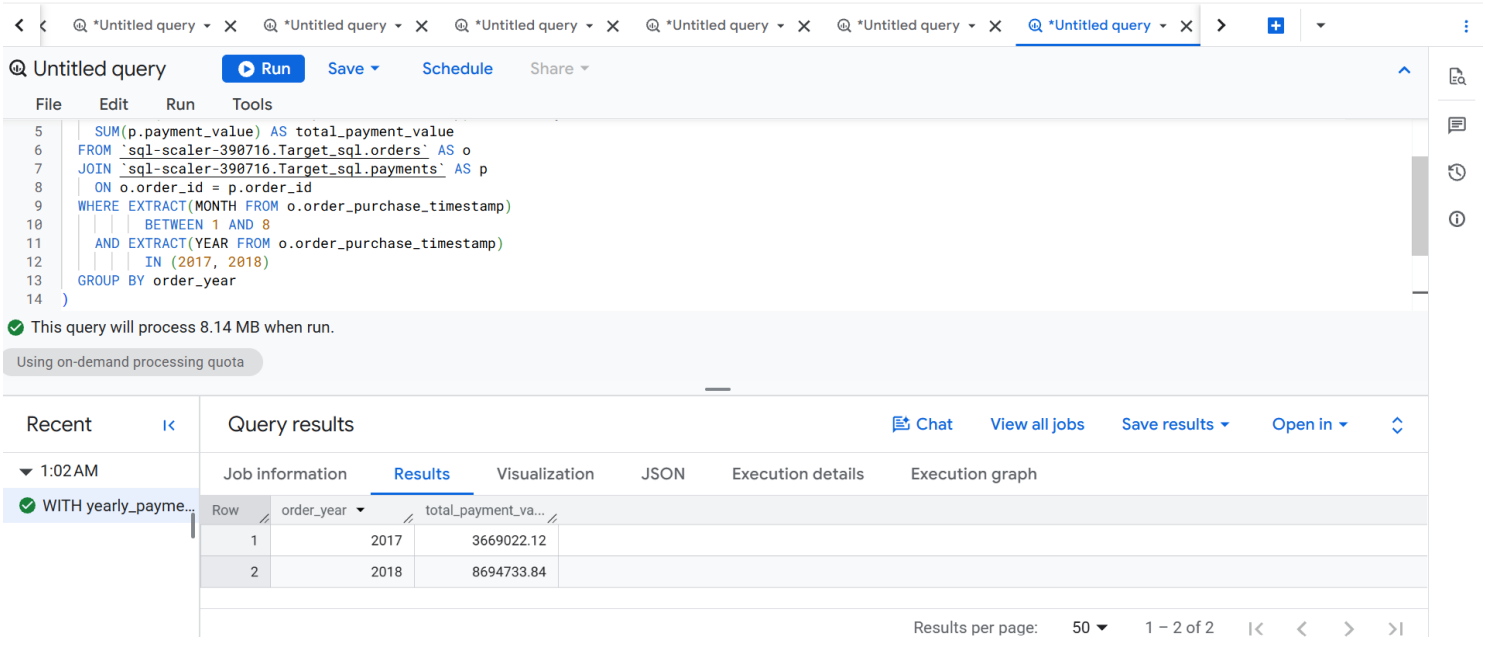
23	RO	253
24	AM	148
25	AC	81
26	AP	68
27	RR	46

4 . Impact on Economy: Analyze the money movement by e-commerce by looking at order prices, freight and others.

1. Get the % increase in the cost of orders from year 2017 to 2018 (include months between Jan to Aug only).
   You can use the "payment_value" column in the payments table to get the cost of orders.
2. Calculate the Total & Average value of order price for each state.
3. Calculate the Total & Average value of order freight for each state.

4a) Query :

```
WITH yearly_payments AS (
 SELECT
  EXTRACT(YEAR FROM o.order_purchase_timestamp) AS order_year,
  SUM(p.payment_value) AS total_payment_value
 FROM `sql-scaler-390716.Target_sql.orders` AS o
 JOIN `sql-scaler-390716.Target_sql.payments` AS p
  ON o.order_id = p.order_id
 WHERE EXTRACT(MONTH FROM o.order_purchase_timestamp)
      BETWEEN 1 AND 8
  AND EXTRACT(YEAR FROM o.order_purchase_timestamp)
      IN (2017, 2018)
 GROUP BY order_year
)
SELECT
 order_year,
 ROUND(total_payment_value, 2) AS total_payment_value
FROM yearly_payments
ORDER BY order_year;
```



4a) Percentage increase calculation (Jan–Aug comparison)

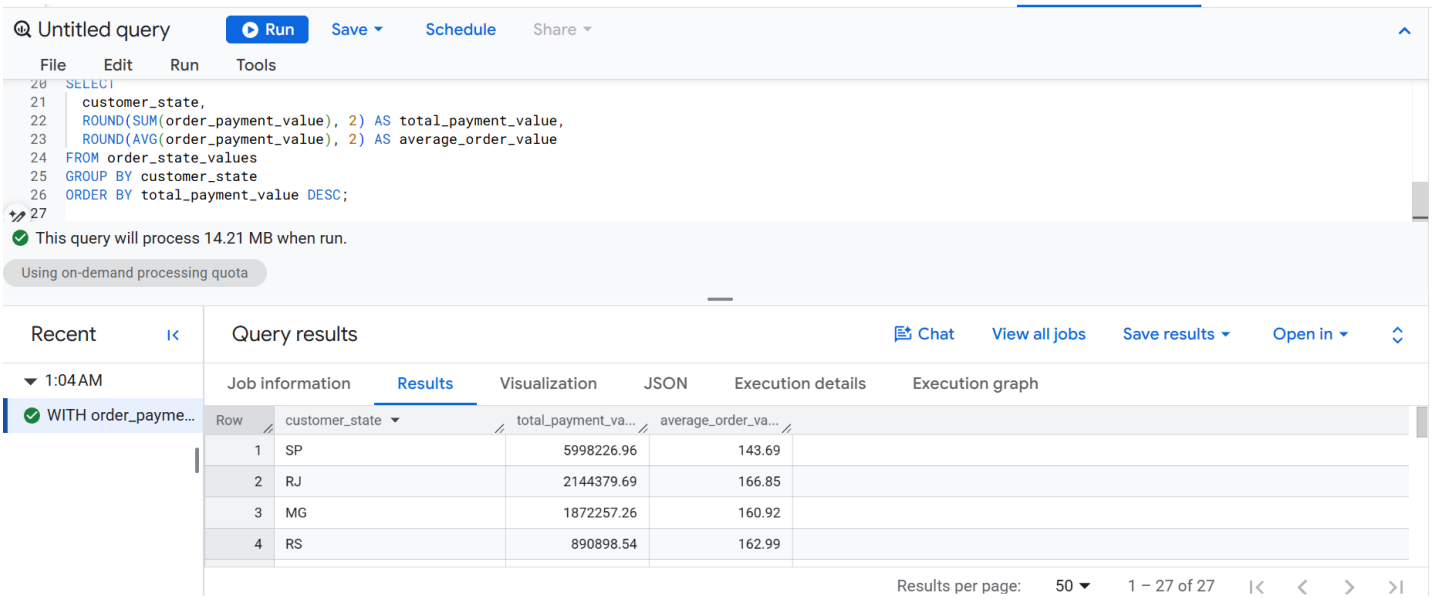
The query above returns payment totals for the two comparable periods. The following query explicitly calculates the absolute and percentage increase.

```
WITH yearly_payments AS (
  SELECT
    EXTRACT(YEAR FROM o.order_purchase_timestamp) AS order_year,
    SUM(p.payment_value) AS total_payment_value
  FROM `sql-scaler-390716.Target_sql.orders` AS o
  JOIN `sql-scaler-390716.Target_sql.payments` AS p
    ON o.order_id = p.order_id
  WHERE EXTRACT(MONTH FROM o.order_purchase_timestamp) BETWEEN 1 AND 8
    AND EXTRACT(YEAR FROM o.order_purchase_timestamp) IN (2017, 2018)
  GROUP BY order_year
),
pivoted AS (
  SELECT
    MAX(IF(order_year = 2017, total_payment_value, NULL)) AS payment_2017_jan_aug,
    MAX(IF(order_year = 2018, total_payment_value, NULL)) AS payment_2018_jan_aug
  FROM yearly_payments
)
SELECT
  ROUND(payment_2017_jan_aug, 2) AS payment_2017_jan_aug,
  ROUND(payment_2018_jan_aug, 2) AS payment_2018_jan_aug,
  ROUND(payment_2018_jan_aug - payment_2017_jan_aug, 2) AS absolute_increase,
  ROUND(SAFE_DIVIDE(payment_2018_jan_aug - payment_2017_jan_aug,
                payment_2017_jan_aug) * 100, 2) AS percentage_increase
FROM pivoted;
```

Documented result: Jan–Aug 2017 = R$ 3,669,022.12; Jan–Aug 2018 = R$ 8,694,733.84; absolute increase = R$ 5,025,711.72; percentage increase = 136.96%. This is payment value, not necessarily net recognised revenue.

4b) Query:

```
WITH order_payments AS (
  SELECT
    order_id,
    SUM(payment_value) AS order_payment_value
  FROM `sql-scaler-390716.Target_sql.payments`
  GROUP BY order_id
),
order_state_values AS (
  SELECT
    c.customer_state,
    o.order_id,
    op.order_payment_value
  FROM `sql-scaler-390716.Target_sql.orders` AS o
  JOIN `sql-scaler-390716.Target_sql.customers` AS c
    ON o.customer_id = c.customer_id
  JOIN order_payments AS op
    ON o.order_id = op.order_id
)
SELECT
  customer_state,
  ROUND(SUM(order_payment_value), 2) AS total_payment_value,
  ROUND(AVG(order_payment_value), 2) AS average_order_value
FROM order_state_values
GROUP BY customer_state
ORDER BY total_payment_value DESC;
```

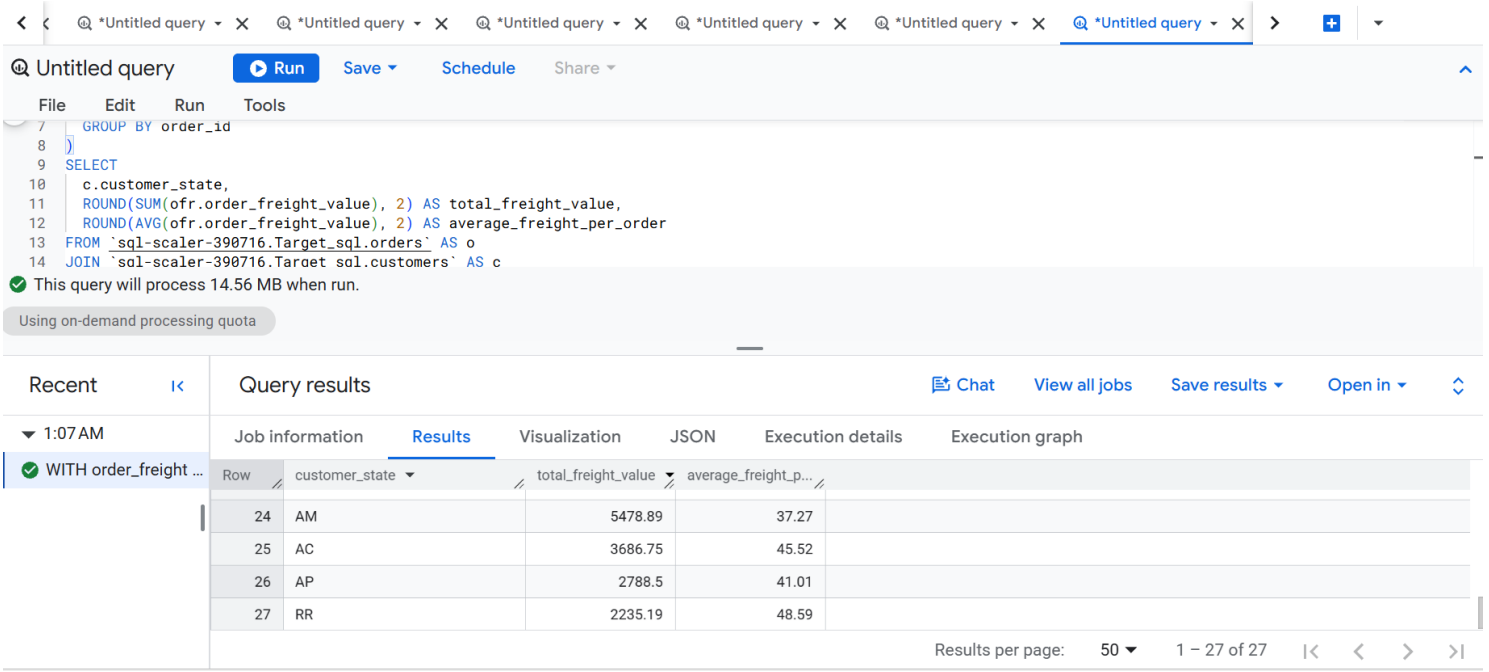


Output :

Row	customer_state	total_payment_value	average_order_value
1	SP	5998226.96	143.69
2	RJ	2144379.69	166.85
3	MG	1872257.26	160.92
4	RS	890898.54	162.99
5	PR	811156.38	160.78
6	SC	623086.43	171.32
7	BA	616645.82	182.44
8	DF	355141.08	165.95
9	GO	350092.31	173.31
10	ES	325967.55	160.34
11	PE	324850.44	196.64
12	CE	279464.03	209.18
13	PA	218295.85	223.89
14	MT	187029.29	206.21
15	MA	152523.02	204.18
16	PB	141545.72	264.08
17	MS	137534.84	192.36
18	PI	108523.97	219.24
19	RN	102718.13	211.79
20	AL	96962.06	234.77
21	SE	75246.25	214.99
22	TO	61485.33	219.59
23	RO	60866.2	240.58
24	AM	27966.93	188.97
25	AC	19680.62	242.97
26	AP	16262.8	239.16
27	RR	10064.62	218.8

4c) Query:

```sql
WITH order_freight AS (
  SELECT
    order_id,
    SUM(freight_value) AS order_freight_value
  FROM `sql-scaler-390716.Target_sql.order_items`
  GROUP BY order_id
)
SELECT
  c.customer_state,
  ROUND(SUM(ofr.order_freight_value), 2) AS total_freight_value,
  ROUND(AVG(ofr.order_freight_value), 2) AS average_freight_per_order
FROM `sql-scaler-390716.Target_sql.orders` AS o
JOIN `sql-scaler-390716.Target_sql.customers` AS c
  ON o.customer_id = c.customer_id
JOIN order_freight AS ofr
  ON o.order_id = ofr.order_id
GROUP BY c.customer_state
ORDER BY total_freight_value DESC;
```



Row	customer_state	total_freight_value	average_freight_p...
24	AM	5478.89	37.27
25	AC	3686.75	45.52
26	AP	2788.5	41.01
27	RR	2235.19	48.59

Output :

Row	customer_state	total_freight_value	average_freight_per_order
1	SP	718723.07	17.37
2	RJ	305589.31	23.95
3	MG	270853.46	23.46
4	RS	135522.74	24.95
5	PR	117851.68	23.58
6	BA	100156.68	29.83
7	SC	89660.26	24.82
8	PE	59449.66	36.07
9	GO	53114.98	26.46
10	DF	50625.5	23.82
11	ES	49764.6	24.58
12	CE	48351.59	36.44
13	PA	38699.3	39.9
14	MA	31523.77	42.6
15	MT	29715.43	32.91
16	PB	25719.73	48.35
17	PI	21218.2	43.04

18	MS	19144.03	27.0
19	RN	18860.1	39.13
20	AL	15914.59	38.72
21	SE	14111.47	40.9
22	TO	11732.68	42.05
23	RO	11417.38	46.22
24	AM	5478.89	37.27
25	AC	3686.75	45.52
26	AP	2788.5 41.01	
27	RR	2235.19	48.59

5 Analysis based on sales, freight, and delivery time.

1.)Find the no. of days taken to deliver each order from the order's purchase date as delivery time.

Also, calculate the difference (in days) between the estimated & actual delivery date of an order.

Do this in a single query.

You can calculate the delivery time and the difference between the estimated & actual delivery date using the given formula:

time_to_deliver = order_delivered_customer_date - order_purchase_timestamp

diff_estimated_delivery = order_estimated_delivery_date - order_delivered_customer_date

2.Find out the top 5 states with the highest & lowest average freight value.

3.Find out the top 5 states with the highest & lowest average delivery time.

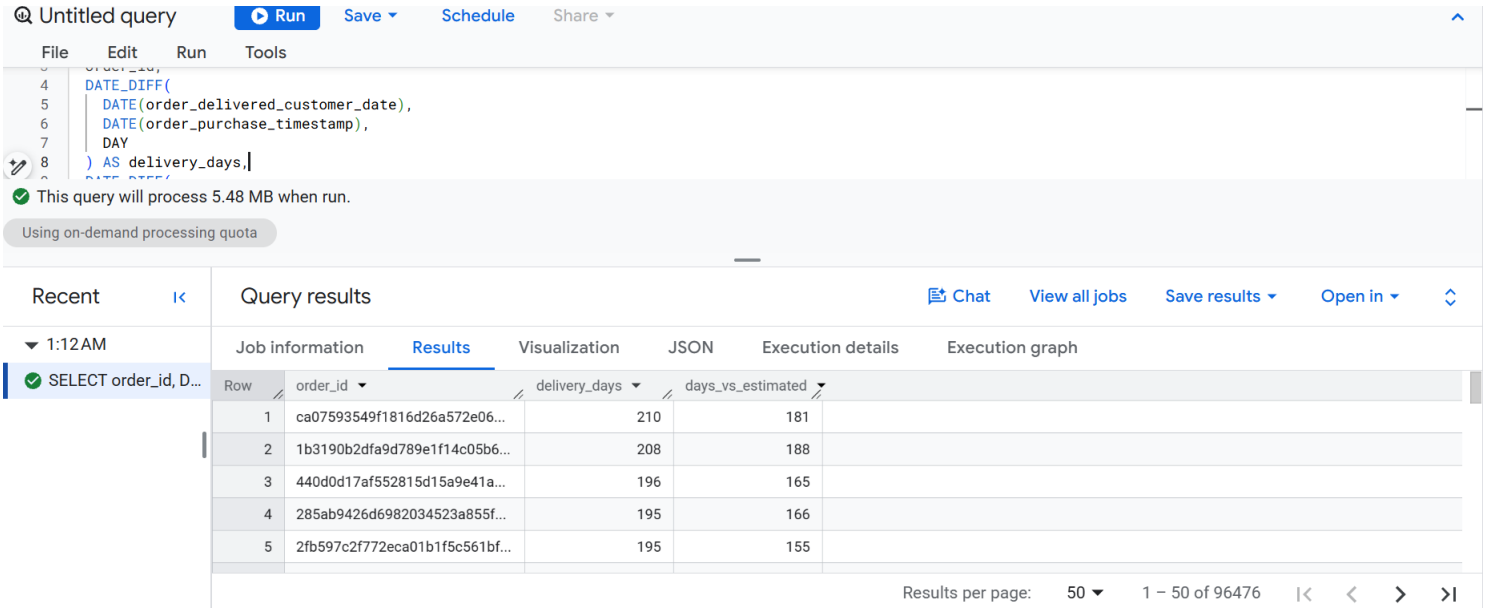
4.Find out the top 5 states where the order delivery is really fast as compared to the estimated date of delivery.

You can use the difference between the averages of actual & estimated delivery date to figure out how fast the delivery was for each state.

5a) Query:

```
SELECT
  order_id,
  DATE_DIFF(
    DATE(order_delivered_customer_date),
    DATE(order_purchase_timestamp),
    DAY
  ) AS delivery_days,
  DATE_DIFF(
    DATE(order_delivered_customer_date),
    DATE(order_estimated_delivery_date),
    DAY
  ) AS days_after_estimated_delivery
FROM `sql-scaler-390716.Target_sql.orders`
WHERE order_purchase_timestamp IS NOT NULL
  AND order_delivered_customer_date IS NOT NULL
  AND order_estimated_delivery_date IS NOT NULL
```

```
ORDER BY delivery_days DESC;
```



Output :

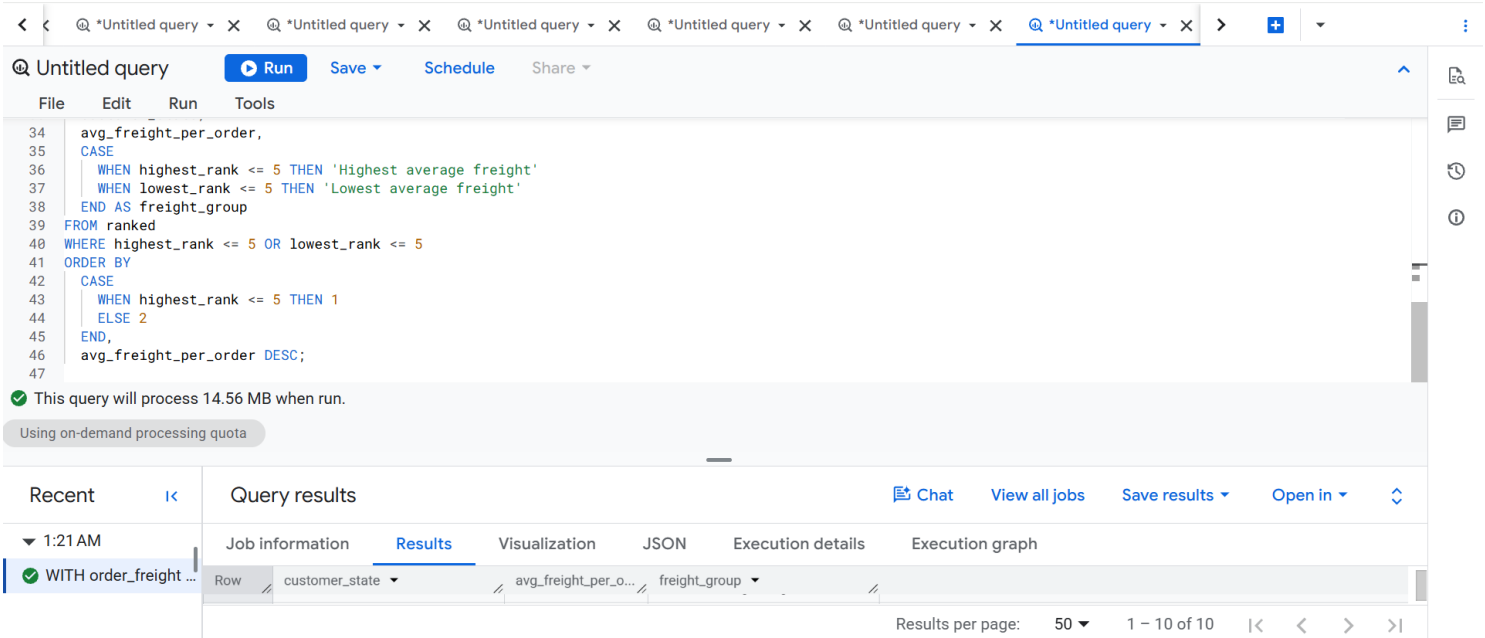
Row	order_id	delivery_days	days_vs_estimated
1	ca07593549f1816d26a572e06dc1eab6	210	181
2	1b3190b2dfa9d789e1f14c05b647a14a	208	188
3	440d0d17af552815d15a9e41abe49359	196	165
4	285ab9426d6982034523a855f55a885e	195	166
5	2fb597c2f772eca01b1f5c561bf6cc7b	195	155
6	0f4519c5f1c541ddec9f21b3bddd533a	194	161
7	47b40429ed8cce3aee9199792275433f	191	175
8	2fe324febf907e3ea3f2aa9650869fa5	190	167
9	c27815f7e3dd0b926b58552628481575	188	162
10	2d7561026d542c8dbd8f0daeadf67a43	188	159
11	437222e3fd1b07396f1d9ba8c15fba59	187	144
12	dfe5f68118c2576143240b8d78e5940a	186	153
13	6e82dcfb5eada6283dba34f164e636f5	183	155
14	2ba1366baecad3c3536f27546d129017	181	152
15	d24e8541128cea179a11a65176e0a96f	175	161
16	3566eabb132f8d64741ae7b921bbd10e	174	137
17	ed8e9faf1b75f43ee027103957135663	173	153
18	2fa29503f2ebd9f53deba187160f3202	172	132
19	a452fba32eab28a4a62af18eed010c0b	168	138

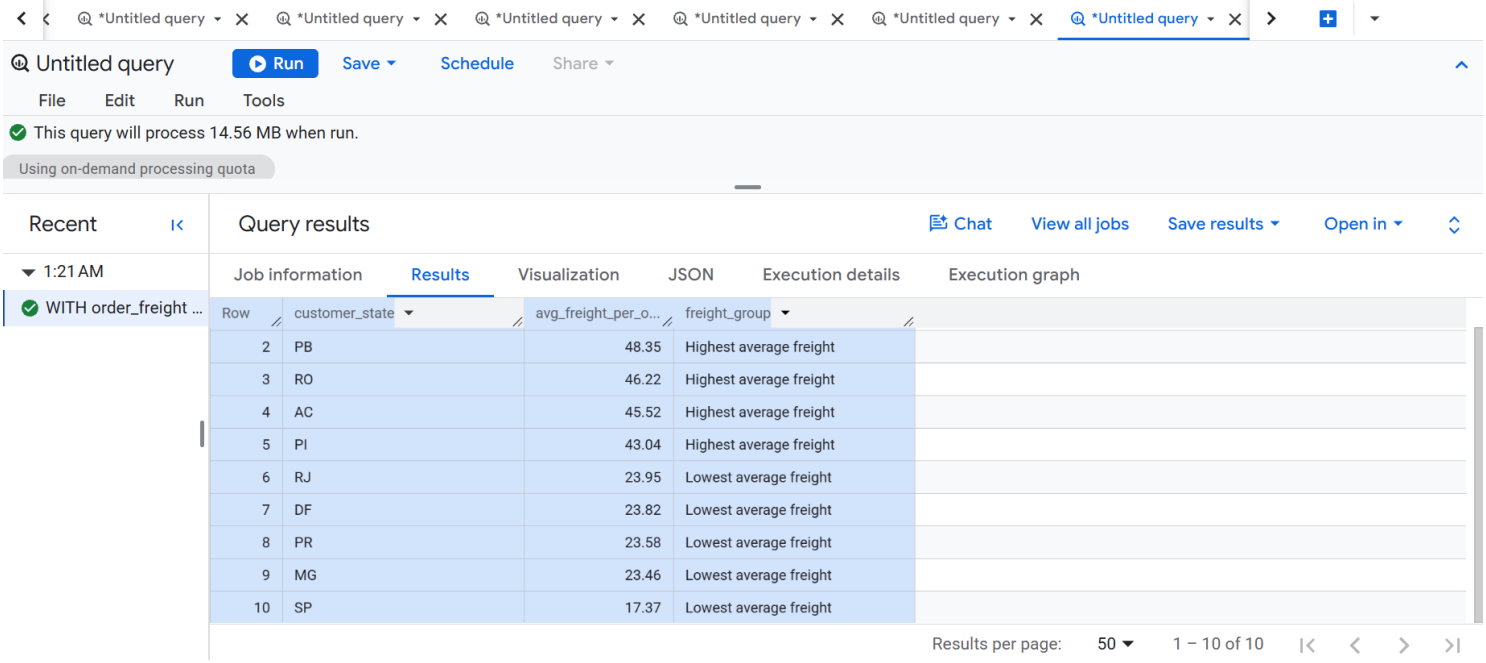
20	525e11b26fdb7f41471d289897d0f6da	168	134	
21	4fbc8d6f2f4db3e789d5a876fa349b56	168	134	
22	df6d8b7768a047c2981bae0a24afbb01	168	132	
23	6e6527028de694ccade37f5a15a6d84a	166	143	
24	031e7d4e559a1bf08e71a419aa998d0a	166	123	
25	cce224811ba8fea016c049693c3e0402	148	126	
26	3c98e4bedff26f850c4f9989b16c8ee7	146	119	
27	0b32bf55276b5feb681f6ed6fd670692	145	121	
28	4f39a94d6e474819d898d6df7d394996	144	112	
29	450cb96c63e1e5b49d34f223f67976d2	143	106	
30	17cc6728043d53cc948551dfbf0a338b	142	110	
31	815b68de0bc7fa6c117e4bcf855734ef	142	111	
32	28f12a3e5117f5e70bbdb45be9188639	139	104	
33	0b7b12a402b064d880d3cb31051f6d29		138	104
34	7e708aed151d6a8601ce8f2eaa712bf4	137	96	
35	c5f43352377291361dad9a7366ae1c89	135	109	
36	29c3b79aace1b72a82b1232bf494e16f	133	94	
37	b31c7dea63bb08f8cdd1ec32514ccf0b	132	109	
38	c2a550cc5f966506b717532441c221e5	131	104	
39	b7624f6b70f7b79ea2b092b6f1be678c	131	96	
40	a6a6c002fd9f0e9eb0c0148448cf8863	130	104	
41	da81fbc27b55e0f3d2813cf2078dc780	127	100	
42	773f048245dc975a5f7c67c2415c88a0	126	96	
43	607a624a0fb32899edc4e3f9c9b24189	125	63	
44	103188b1afa4614c023b86344f79727b	119	105	
45	e7fc47a9bd5fd8c1d1b566d1553be0f4	117	104	
46	5fc65d39393a0107a062195fdc96ad34	115	97	
47	e7b87b156cde9f1d3a765da80d093d6a	110	88	
48	a4efaffc506a395c9cea7402b078c1e5	110	73	
49	e52c9dfec957c503bed5d050a39c740f	108	78	
50	3b919fbc512d96f81330d2aae3ed0cf6	108	79	

5b) Query:

```
WITH order_freight AS (
  SELECT
    order_id,
    SUM(freight_value) AS order_freight_value
```

```sql
  FROM `sql-scaler-390716.Target_sql.order_items`
  GROUP BY order_id
),
state_freight AS (
  SELECT
    c.customer_state,
    AVG(ofr.order_freight_value) AS avg_freight_per_order
  FROM `sql-scaler-390716.Target_sql.orders` AS o
  JOIN `sql-scaler-390716.Target_sql.customers` AS c
    ON o.customer_id = c.customer_id
  JOIN order_freight AS ofr
    ON o.order_id = ofr.order_id
  GROUP BY c.customer_state
),
ranked AS (
  SELECT
    customer_state,
    ROUND(avg_freight_per_order, 2) AS avg_freight_per_order,
    ROW_NUMBER() OVER (
      ORDER BY avg_freight_per_order DESC
    ) AS highest_rank,
    ROW_NUMBER() OVER (
      ORDER BY avg_freight_per_order ASC
    ) AS lowest_rank
  FROM state_freight
)
SELECT
  customer_state,
  avg_freight_per_order,
  CASE
    WHEN highest_rank <= 5 THEN 'Highest average freight'
    WHEN lowest_rank <= 5 THEN 'Lowest average freight'
  END AS freight_group
FROM ranked
WHERE highest_rank <= 5 OR lowest_rank <= 5
ORDER BY
  CASE
    WHEN highest_rank <= 5 THEN 1
    ELSE 2
  END,
  avg_freight_per_order DESC;
```

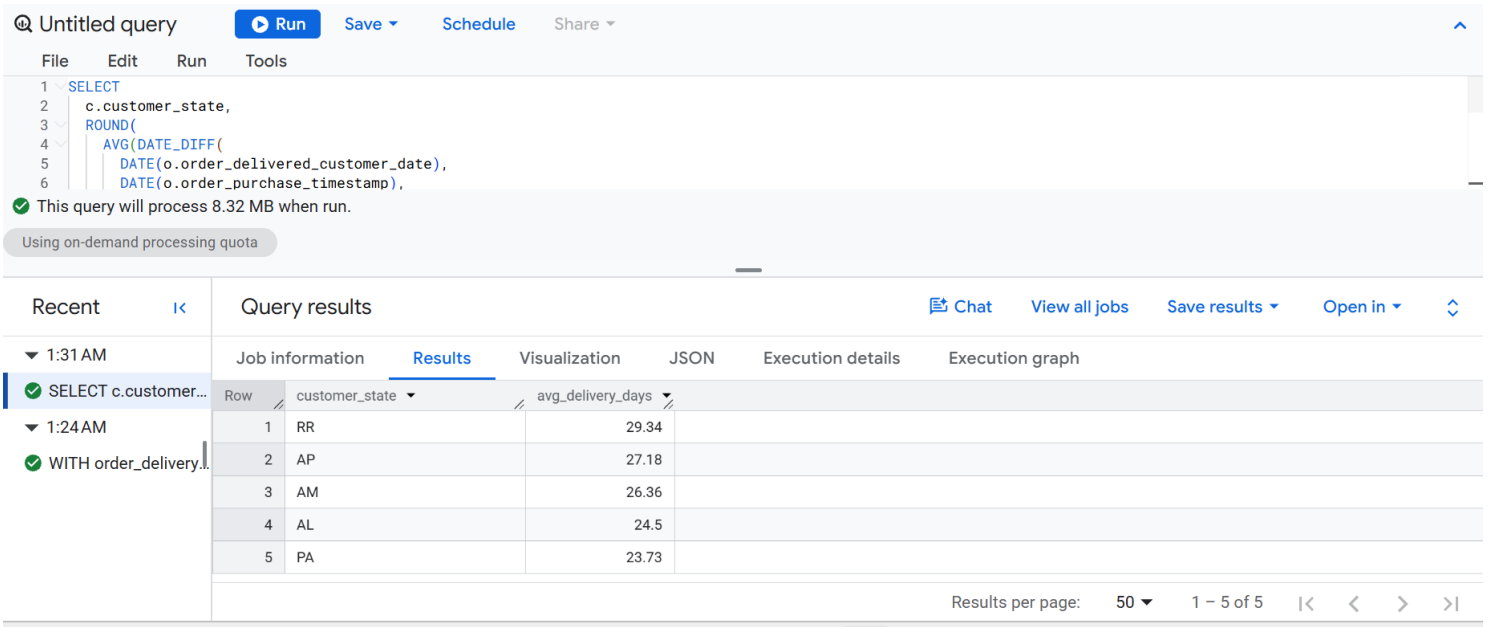




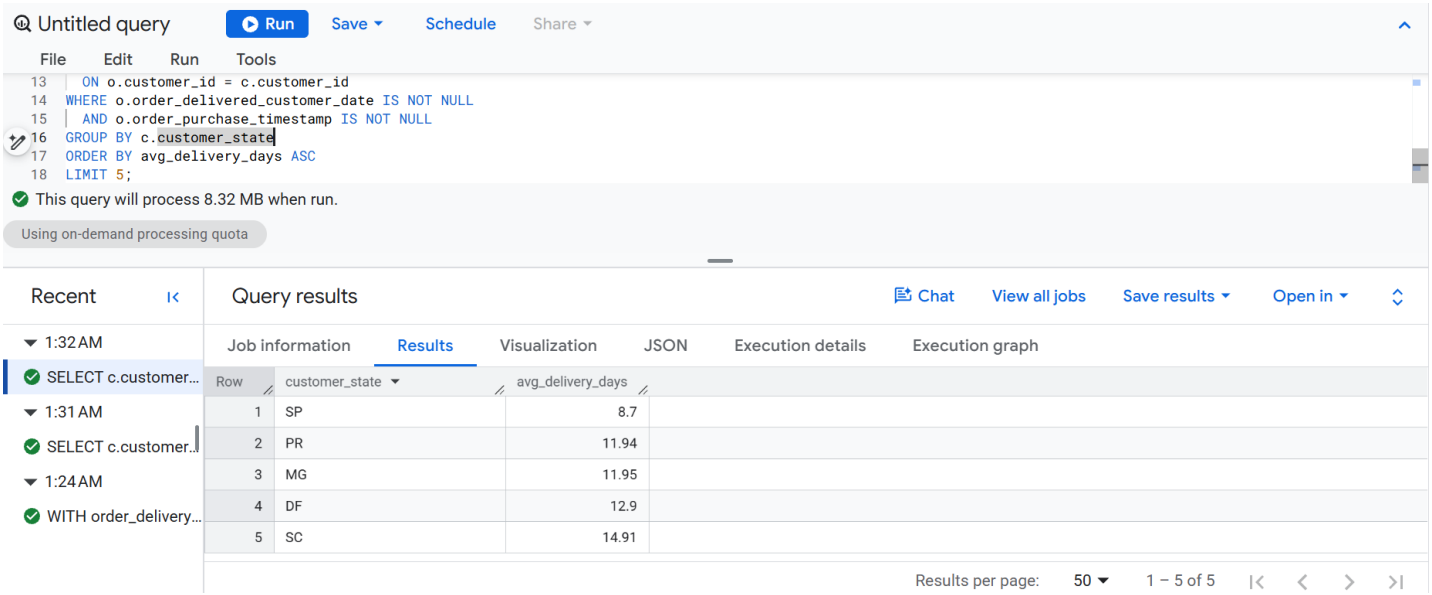
5c) Query:

Highest average delivery time :

```
SELECT
  c.customer_state,
  ROUND(
    AVG(DATE_DIFF(
      DATE(o.order_delivered_customer_date),
      DATE(o.order_purchase_timestamp),
      DAY
    )),
    2
  ) AS avg_delivery_days
FROM `sql-scaler-390716.Target_sql.orders` AS o
JOIN `sql-scaler-390716.Target_sql.customers` AS c
  ON o.customer_id = c.customer_id
WHERE o.order_delivered_customer_date IS NOT NULL
  AND o.order_purchase_timestamp IS NOT NULL
GROUP BY c.customer_state
ORDER BY avg_delivery_days DESC
LIMIT 5;
```



Lowest average delivery time :



Lowest average delivery time — query and result

```
SELECT
  c.customer_state,
  ROUND(
    AVG(DATE_DIFF(
      DATE(o.order_delivered_customer_date),
      DATE(o.order_purchase_timestamp),
      DAY
    )),
    2
  ) AS avg_delivery_days
FROM `sql-scaler-390716.Target_sql.orders` AS o
JOIN `sql-scaler-390716.Target_sql.customers` AS c
  ON o.customer_id = c.customer_id
WHERE o.order_delivered_customer_date IS NOT NULL
  AND o.order_purchase_timestamp IS NOT NULL
GROUP BY c.customer_state
ORDER BY avg_delivery_days ASC
LIMIT 5;
```

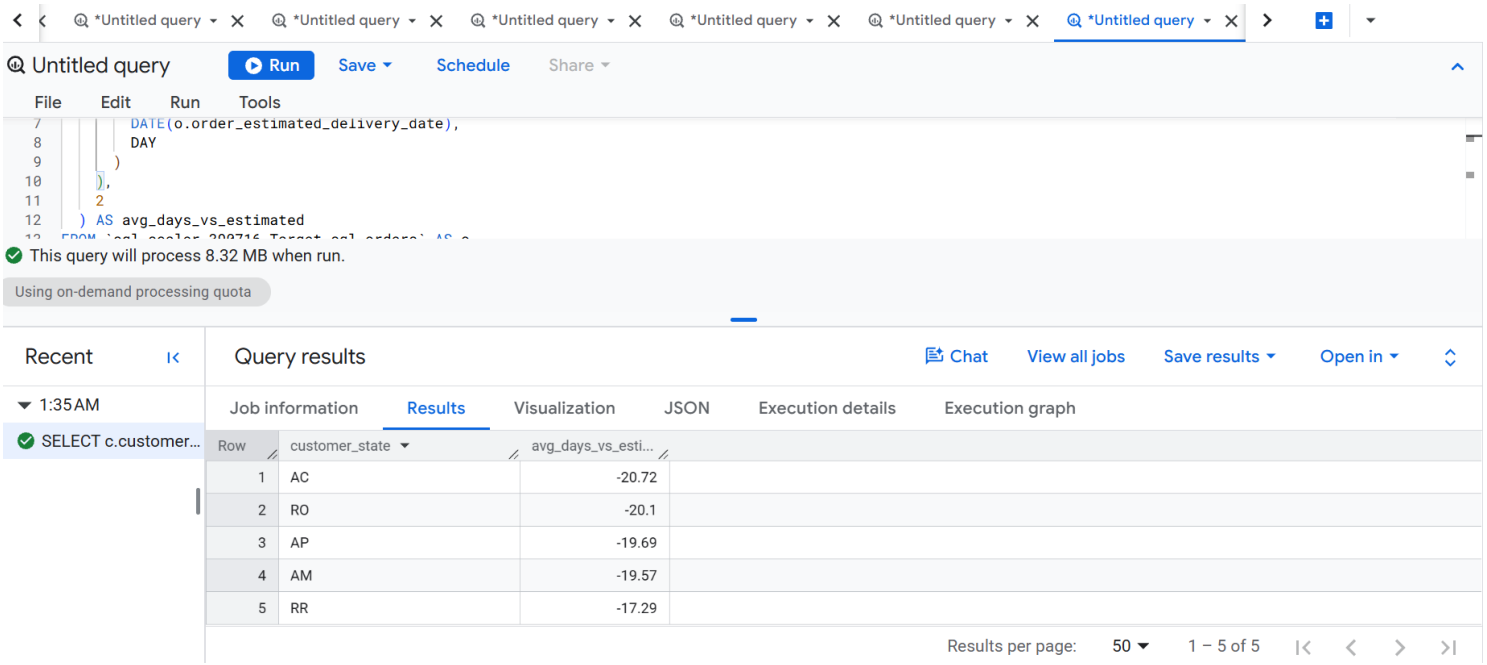
Documented lowest average delivery times: RR — 29.34 days; AP — 27.18 days; AM — 26.36 days; AL — 24.50 days; PA — 23.73 days. These averages include delivered orders only.

Delivery-difference interpretation

The SQL uses actual delivery date minus estimated delivery date. Positive values mean delivery occurred after the estimated date (late); negative values mean delivery occurred before the estimated date (early); zero means delivery occurred on the estimated date.

5d) Query:

```
SELECT
 c.customer_state,
 ROUND(
  AVG(
   DATE_DIFF(
    DATE(o.order_delivered_customer_date),
    DATE(o.order_estimated_delivery_date),
    DAY
   )
  ),
  2
 ) AS avg_days_vs_estimated
FROM `sql-scaler-390716.Target_sql.orders` AS o
JOIN `sql-scaler-390716.Target_sql.customers` AS c
 ON o.customer_id = c.customer_id
WHERE o.order_delivered_customer_date IS NOT NULL
 AND o.order_estimated_delivery_date IS NOT NULL
GROUP BY c.customer_state
ORDER BY avg_days_vs_estimated ASC
LIMIT 5;
```



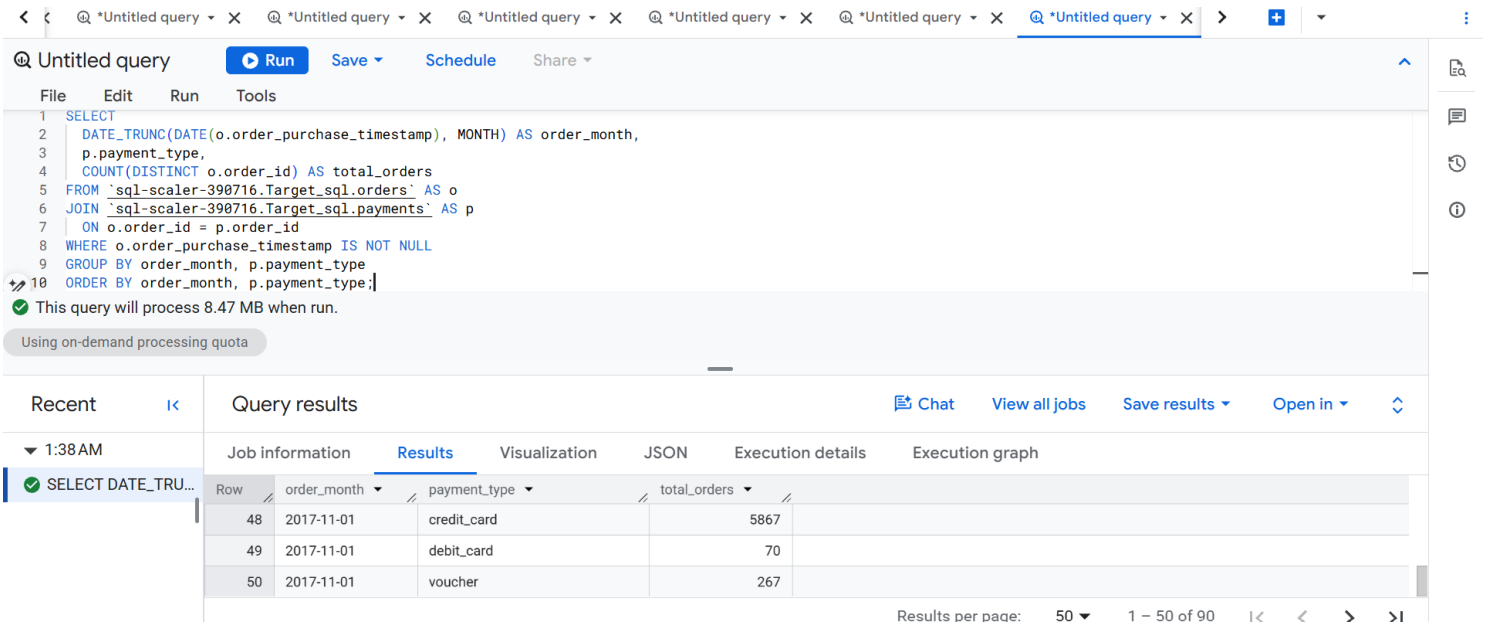
6. **Analysis based on the payments:**

a) Find the month-on-month no. of orders placed using different payment types.

b) Find the no. of orders placed on the basis of the payment installments that have been paid.

6a) Query:

```
SELECT
  DATE_TRUNC(DATE(o.order_purchase_timestamp), MONTH) AS order_month,
  p.payment_type,
  COUNT(DISTINCT o.order_id) AS total_orders
FROM `sql-scaler-390716.Target_sql.orders` AS o
JOIN `sql-scaler-390716.Target_sql.payments` AS p
  ON o.order_id = p.order_id
WHERE o.order_purchase_timestamp IS NOT NULL
GROUP BY order_month, p.payment_type
ORDER BY order_month, p.payment_type;
```



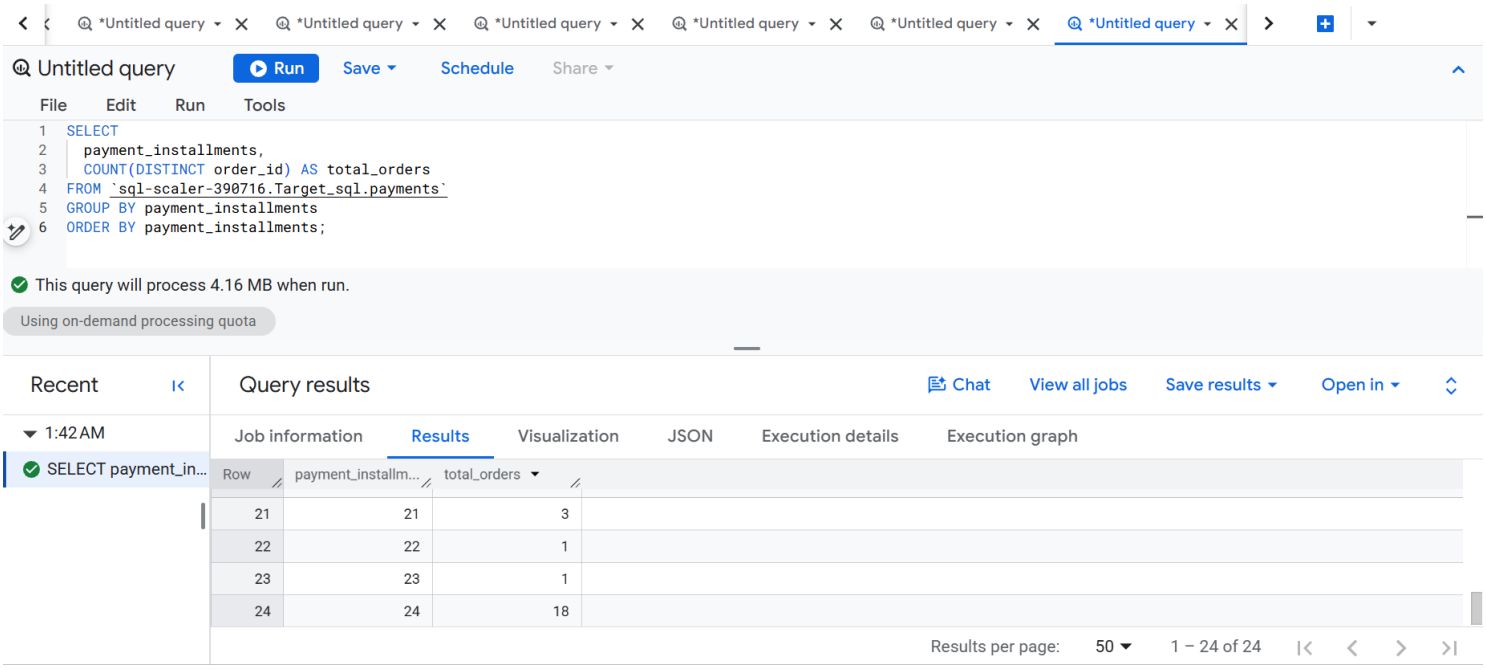
Output :

Q6(a) result interpretation

The query groups distinct orders by purchase month and payment type. The results reviewed showed credit card as the most frequently represented payment type. An order can appear under more than one payment type if it has multiple payment records. The full result table was not present in the uploaded report, so exact monthly counts are not reproduced here.

6b)Query:

```
SELECT
  payment_installments,
  COUNT(DISTINCT order_id) AS total_orders
FROM `sql-scaler-390716.Target_sql.payments`
GROUP BY payment_installments
ORDER BY payment_installments;
```



Output :

Installments	Total orders
0	2
1	49,060
2	12,389
3	10,443
4	7,088
5	5,234
6	3,916
7	1,623
8	4,253
9	644
10	5,315
11	23
12	133
13	16
14	15
15	74
16	5
17	8
18	27
20	17
21	3
22	1
23	1
24	18